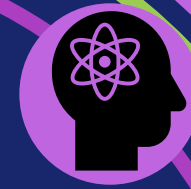
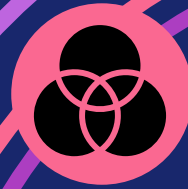


2026年度 前期 データマイニング特論 (I20教室)
第6回 2026年5月21日(木) 13:10~14:40



データマイニング特論

第6回

アンサンブル学習

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1



アンサブル学習とは

本日の学習内容を確認しよう

【前回の復習】 1 本の決定木の限界

>>> airbnbの収集データにおけるスーパーホスト（SH）を分類
決定木（max_depth=4）の検証精度 = 0.716、SH再現率 = 0.502

不安定性 : 学習データが少し変わると木の構造が大きく変わる。
⇒決定木は「高バリエーションなデータに敏感過ぎる」問題。

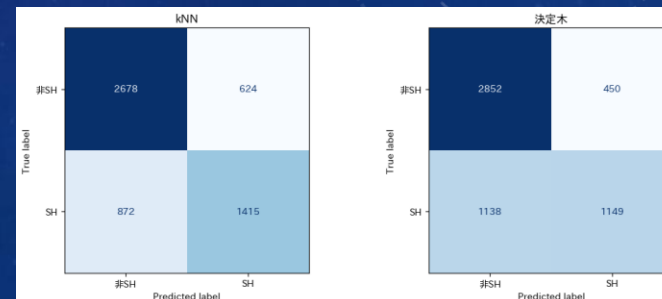
過学習リスク : 深い木はデータを丸暗記する傾向がある。
⇒max_depthで制御しても限界がある。

局所最適 : 貪欲法（greedy）で分岐を決めるため、
全体最適を保証できない。

解決策として、
「弱い学習器をたくさん集めて投票させる」
→ これがアンサンブル学習の核心

kNN				
	precision	recall	f1-score	support
非SH	0.754	0.811	0.782	3302
SH	0.694	0.619	0.654	2287
accuracy			0.732	5589
macro avg	0.724	0.715	0.718	5589
weighted avg	0.730	0.732	0.730	5589

決定木				
	precision	recall	f1-score	support
非SH	0.715	0.864	0.782	3302
SH	0.719	0.502	0.591	2287
accuracy			0.716	5589
macro avg	0.717	0.683	0.687	5589
weighted avg	0.716	0.716	0.704	5589



「多数決の知恵」が、なぜ束ねると強くなるのか？

>>> 1906年、フランシス・ゴルトンの実験：800人が牛の体重を予測

個々の予測はバラバラで外れていたが、800人の中央値は1207ポンドだった。

⇒実際の体重は1198ポンド。誤差わずか0.8%！

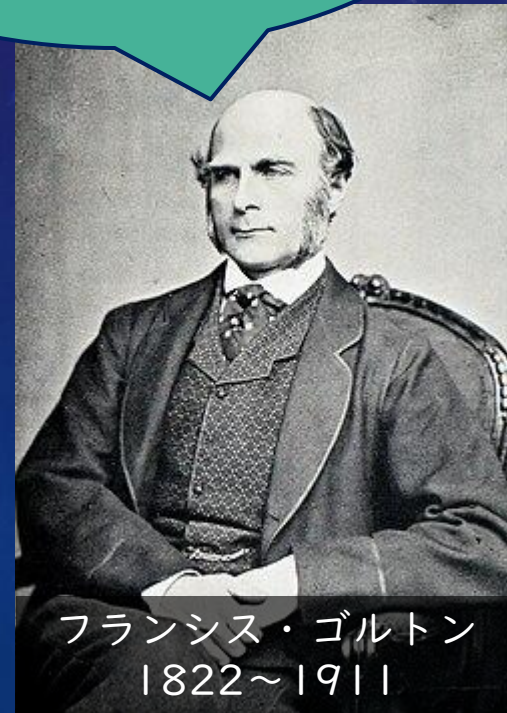
個々の「偏り」が互いに打ち消し合うことで、集合知は驚くほど正確になる。

これが「アンサンブル学習」の直感。
弱い分類器を多数集めれば、強い分類器に変わる。

素人の予想はバラバラで当たらないだろう。

だが、一人一人は不完全でも、多くの独立した判断を集めると、非常に正確になることがある。

ボクの体重を当てられるかな？



バイアス・バリエーションのトレードオフ

>>> 予測誤差 = バイアス² + バリエーション + 不可避な誤差（ノイズ）

バリエーション (Variance)

⇒ モデルの「ブレやすさ」

⇒ 訓練データに敏感すぎると汎化性能が下がる（深い決定木 → 高バリエーション）

汎化性能：AIや機械学習モデルが、学習に使用していない「未知のデータ」に対してどれだけ正確に予測や判断ができるかを示す能力

⇒ バギングが低減

バイアス (Bias)

⇒ モデルの「系統的なズレ」

⇒ モデルが単純すぎると真の関係を捉えられない（浅い決定木 → 高バイアス）

⇒ ブースティングが低減

本日の環境準備（Ⅰ）

いろいろ試してみよう！

```
# 日本語対応のモジュールをインストールする(その1)
```

```
!pip install japanize-matplotlib
```

```
# 日本語対応のモジュールをインストールする(その2)
```

```
!apt-get install -y fonts-ipafont-gothic
```

```
# 各ライブラリーをインポートする
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.patches as mpatches
```

```
import japanize_matplotlib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import (  
    accuracy_score, classification_report,  
    confusion_matrix, ConfusionMatrixDisplay, f1_score  
)
```

```
import xgboost as xgb
```

```
print("✓ ライブラリーのインポート完了")
```

```
print(f" XGBoost version: {xgb.__version__}")
```


本日の環境準備（２）

いろいろ試してみよう！

#データ読み込み

```
URL = "https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/2025-09-29/data/listings.csv.gz"
print("データを読み込み中...")
df = pd.read_csv(URL, compression="gzip", low_memory=False)
print(f"読み込み完了:{df.shape[0]:,} 行 × {df.shape[1]} 列")
```

ターゲット変数の作成

```
df["is_superhost"] = (df["host_is_superhost"] == "t").astype(int)
```

価格列の前処理

```
if df["price"].dtype == object:
    df["price"] = df["price"].str.replace(r"[$,]", "", regex=True).astype(float)
```

特徴量の選択

```
FEATURES = [
    "price", "number_of_reviews", "review_scores_rating",
    "accommodates", "bedrooms", "minimum_nights", "availability_365",
]
df_clean = df[FEATURES + ["is_superhost"]].dropna()
print(f"前処理後:{df_clean.shape[0]:,} 行")
print(f"スーパーホスト比率:{df_clean['is_superhost'].mean():.1%}")
X = df_clean[FEATURES].values
y = df_clean["is_superhost"].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"ℳn訓練データ:{len(X_train):,} 件 テストデータ:{len(X_test):,} 件")
```

```
データを読み込み中...
読み込み完了: 27,945 行 × 79 列
前処理後: 21,977 行
スーパーホスト比率: 44.4%
```

```
訓練データ: 17,581 件 テストデータ: 4,396 件
```


本日の環境準備（3）

いろいろ試してみよう！

ベースライン(決定木)

```
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

print(f"テスト正解率:{accuracy_score(y_test, y_pred_dt):.3f}")
print(f"SH F1スコア :{f1_score(y_test, y_pred_dt):.3f}")
print(f"SH 再現率 :{classification_report(y_test, y_pred_dt, output_dict=True)['1']['recall']:.3f}")
```

テスト正解率 : 0.725
SH F1スコア : 0.680
SH 再現率 : 0.658

2

バギング (Bagging)

並列に束ねる



バギング (Bootstrap AGGregatING)

>>> Bootstrap Aggregating — 「データを少しずつ変えながら同時に学習させる」

① ブートストラップ

⇒ 元データからランダムに復元抽出 (約63%が選ばれ37%は重複する)

選ばれる確率 : $1 - 1/e \doteq 0.63$

選ばれない確率 : $1/e \doteq 0.37$

② 独立した学習

⇒ それぞれ異なるデータで決定木を学習 (並列処理可能)

③ 多数決で統合

⇒ 分類 : 各木の多数決

回帰 : 各木の平均値

効果 : 各木が異なるデータで学習するため「高バリエーション問題」が解消される

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale with degree markings (40, 150, 160, 170, 180, 190, 210, 220, 230, 240, 250, 260) is visible on the left side. Other circular patterns, some with arrows indicating direction, are scattered across the image.

ランダムフォレスト

ランダムフォレスト (1)

>>> ランダムフォレスト — バギング + 特徴量のランダム化

バギングからの進化

①特徴量もランダムに選ぶ

- ⇒ 各分岐で、全特徴量ではなくランダムなサブセット ($\sqrt{p}$ 個) から最良の分割を選択する。
- ⇒ 木同士の相関を下げて「多様性」を確保する

②Out-of-Bag (OOB) 評価

- ⇒ 学習に使わなかった約37%のデータで自動的に評価できる。
- ⇒ テストデータなしでも汎化性能が推定可能。
- ⇒ 交差検証が不要

重要なハイパーパラメータ

- `n_estimators` (木の本数) : 多いほど安定するが計算コスト増。通常500~1000。
- `mtry` (各分岐で試す特徴量数) : 分類は $\sqrt{p}$ 、回帰は $p/3$ が目安。

ランダムフォレスト (2)

いろいろ試してみよう！

#ランダムフォレスト

```
rf = RandomForestClassifier(  
    n_estimators=500,    # 木の本数  
    max_features="sqrt", # 各分岐で試す特徴量数(vp)  
    random_state=42,  
    n_jobs=-1,          # 並列処理(全コア使用)  
    oob_score=True,     # OOBスコアを計算  
)  
print("学習中...")  
rf.fit(X_train, y_train)  
y_pred_rf = rf.predict(X_test)  
  
print(f"テスト正解率:{accuracy_score(y_test, y_pred_rf):.3f}")  
print(f"SH F1スコア :{f1_score(y_test, y_pred_rf):.3f}")  
print(f"OOBスコア :{rf.oob_score_: .3f}  ← テストデータなしで推定した精度")
```

#次に続く。。。

ランダムフォレスト (3)

いろいろ試してみよう！

n_estimatorsとOOBエラーの関係を確認

print("nOOBエラー率の推移(木の本数が増えると安定する):")

```
rf_oob = RandomForestClassifier(
    n_estimators=1, warm_start=True, oob_score=True,
    max_features="sqrt", random_state=42
)
```

oob_errors = []

ns = [1, 5, 10, 20, 50, 100, 200, 300, 500]

for n in ns:

rf_oob.n_estimators = n

rf_oob.fit(X_train, y_train)

oob_errors.append(1 - rf_oob.oob_score_)

fig, ax = plt.subplots(figsize=(8, 4))

ax.plot(ns, oob_errors, "o-", color="#27AE60", linewidth=2, markersize=6)

ax.set_xlabel("木の本数(n_estimators)", fontsize=12)

ax.set_ylabel("OOBエラー率", fontsize=12)

ax.set_title("木の本数とOOBエラー率の関係", fontsize=13)

ax.grid(True, alpha=0.3)

ax.axvline(x=100, color="red", linestyle="--", alpha=0.5, label="100本で概ね収束")

ax.legend(fontsize=10)

plt.tight_layout()

plt.savefig("rf_oob_error.png", dpi=150, bbox_inches="tight")

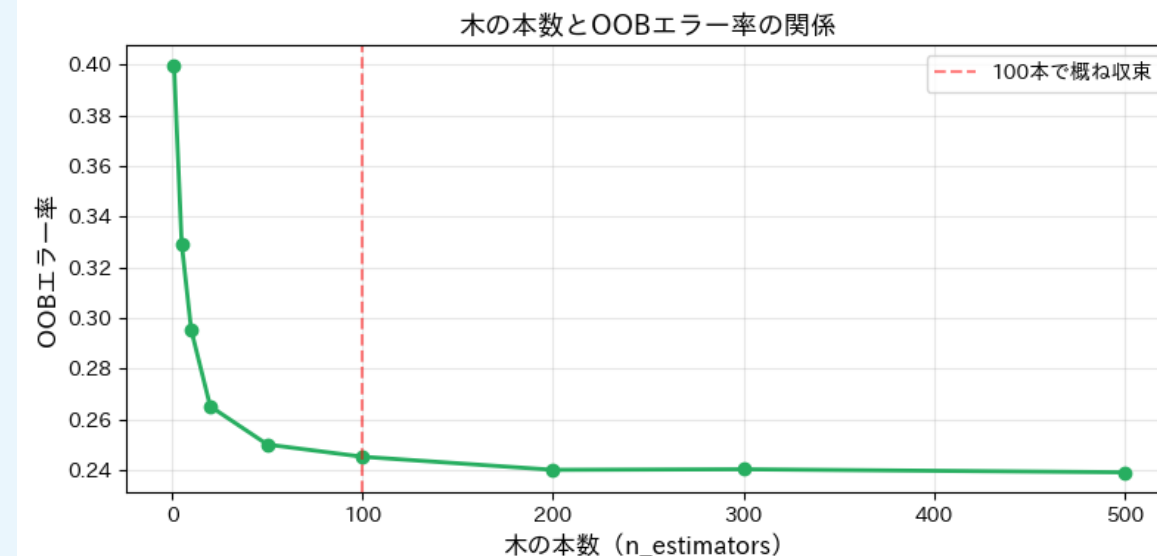
plt.show()

テスト正解率: 0.755

SH F1スコア : 0.718

OOBスコア : 0.761 ← テストデータなしで推定した精度

OOBエラー率の推移(木の本数が増えると安定する):



3

ブースティング (Boosting)

逐次的に弱点を克服する



ブースティング

>>> バギングとの決定的な違い：「前の木の失敗を次の木が修正する」 逐次学習

Round 1

⇒全データに均等な重みで最初の弱い木を学習。

Round 2

⇒Round 1 で間違えたデータの重みを増やして次の木を学習。

Round 3

⇒前回の残差（誤り）に集中して学習。得意分野が異なる木が増える。

Round 4

⇒各木に「精度に応じた重み」をつけて加重投票。

→ 強力なモデルに！

効果： 前の木が苦手なデータを次の木が補うため「高バイアス問題」を解消できる

The background is a gradient of dark blue and purple, speckled with small white dots. Overlaid on this are several faint, white geometric patterns. A large circular scale with degree markings from 140 to 260 is prominent on the left. Other elements include concentric circles, dashed lines, and curved arrows, creating a technical or scientific aesthetic.

XGBoost

XGBoost (I)

>>> eXtreme Gradient Boosting —

勾配ブースティングを極限まで高速化・高精度化したフレームワーク

①正則化項（ペナルティー項）

⇒過学習を防ぐL1・L2正則化が組み込まれており、
ただのブースティングより汎化性能が高い。

②欠損値処理

⇒欠損値を自動で処理する機能を持ち、前処理の手間を大幅に削減できる。

③高速計算

⇒並列処理・キャッシュ最適化・スパースデータへの対応で従来のGBDTより
大幅に高速。

④スケーラビリティ

⇒大規模データ・分散処理にも対応。KaggleなどML競技で最も使われる手法。

XGBoost (2)

いろいろ試してみよう！

```
# XGBoost
xgb_model = xgb.XGBClassifier(
    n_estimators=200,      # 木の本数
    learning_rate=0.1,     # 学習率(eta)
    max_depth=4,          # 木の深さ
    subsample=0.8,        # データのサブサンプル比率
    colsample_bytree=0.8,  # 特徴量のサブサンプル比率
    reg_lambda=1.0,       # L2正則化
    use_label_encoder=False,
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1,
)
print("学習中...")
xgb_model.fit(
    X_train, y_train,
    eval_set=[(X_train, y_train), (X_test, y_test)],
    verbose=False,
)
y_pred_xgb = xgb_model.predict(X_test)

print(f"テスト正解率:{accuracy_score(y_test, y_pred_xgb):.3f}")
print(f"SH F1スコア :{f1_score(y_test, y_pred_xgb):.3f}")
```

#次に続く。。。

XGBoost (3)

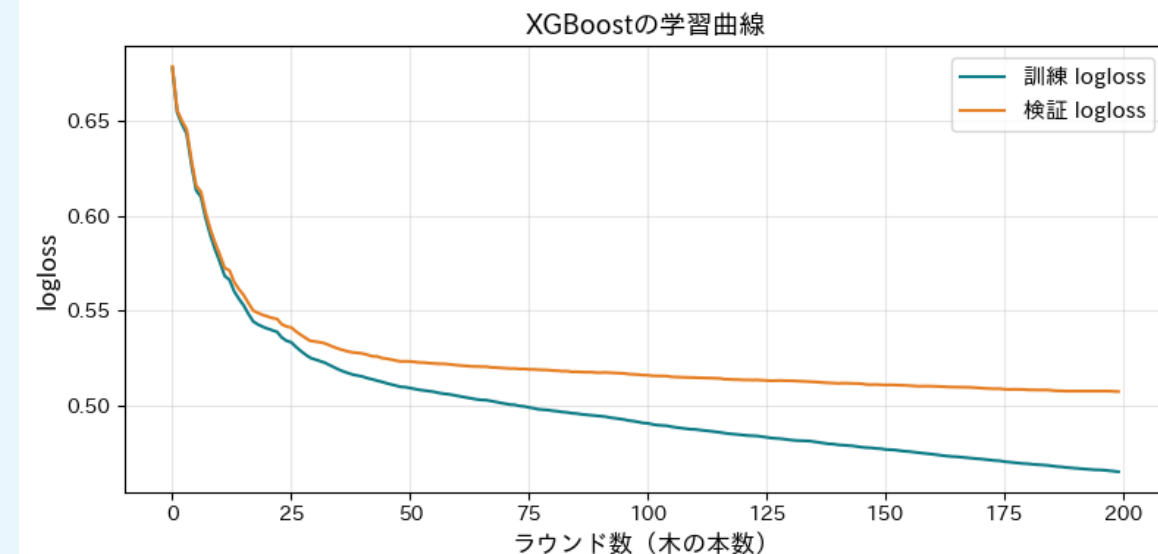
いろいろ試してみよう！

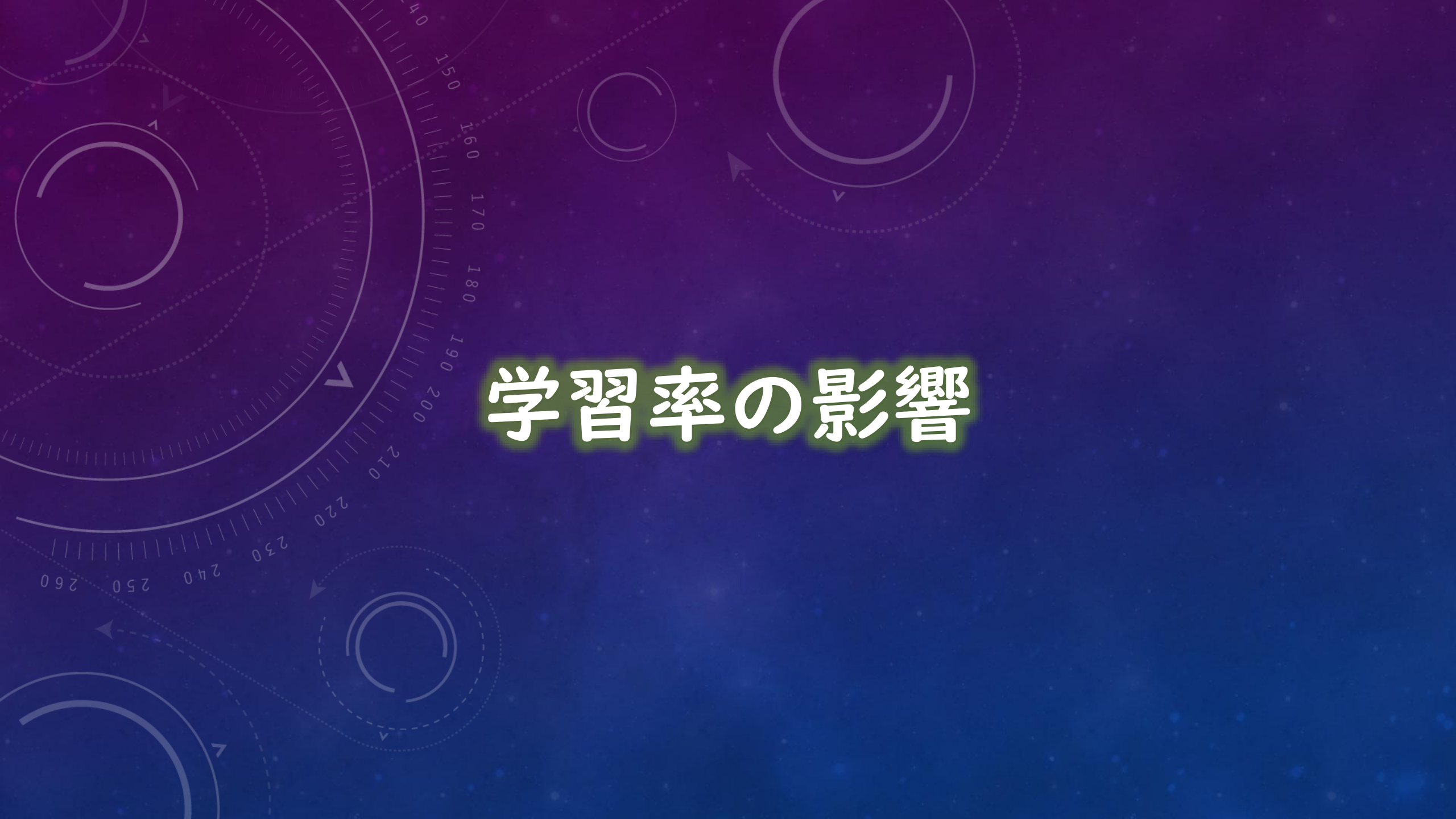
学習曲線(訓練 vs 検証 logloss)

```
results = xgb_model.evals_result()
epochs = len(results["validation_0"]["logloss"])
x_axis = range(epochs)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x_axis, results["validation_0"]["logloss"],
        label="訓練 logloss", color="#0E7C86", linewidth=1.5)
ax.plot(x_axis, results["validation_1"]["logloss"],
        label="検証 logloss", color="#E67E22", linewidth=1.5)
ax.set_xlabel("ラウンド数(木の本数)", fontsize=12)
ax.set_ylabel("logloss", fontsize=12)
ax.set_title("XGBoostの学習曲線", fontsize=13)
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("xgb_learning_curve.png", dpi=150, bbox_inches="tight")
plt.show()
```

テスト正解率 : 0.751
SH F1スコア : 0.711



The background is a deep blue gradient with a subtle pattern of white dots. Overlaid on this are several concentric circles and arcs in a lighter blue color. Some of these arcs have degree markings, such as 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, and 260. There are also small white arrows pointing in various directions, suggesting a sense of rotation or movement. The overall aesthetic is technical and modern.

学習率の影響

学習率 (eta) の影響

いろいろ試してみよう！

学習曲線(訓練 vs 検証 logloss)

```
lr_configs = [  
    {"lr": 0.30, "n": 100, "color": "#C0392B", "label": "eta=0.30(速い・不安定)"},  
    {"lr": 0.10, "n": 200, "color": "#E67E22", "label": "eta=0.10(バランス)"},  
    {"lr": 0.05, "n": 400, "color": "#27AE60", "label": "eta=0.05(遅い・安定)"},  
    {"lr": 0.01, "n": 1000, "color": "#0E7C86", "label": "eta=0.01(非常に遅い)"},  
]
```

```
results_lr = []  
for cfg in lr_configs:  
    m = xgb.XGBClassifier(  
        n_estimators=cfg["n"], learning_rate=cfg["lr"],  
        max_depth=4, subsample=0.8, colsample_bytree=0.8,  
        use_label_encoder=False, eval_metric="logloss",  
        random_state=42, n_jobs=-1, verbosity=0  
    )  
    m.fit(X_train, y_train, verbose=False)  
    pred = m.predict(X_test)  
    results_lr.append({  
        "label": cfg["label"],  
        "accuracy": accuracy_score(y_test, pred),  
        "f1": f1_score(y_test, pred),  
        "n": cfg["n"],  
        "lr": cfg["lr"],  
        "color": cfg["color"],  
    })  
print(f" {cfg['label']:30s} → 精度:{accuracy_score(y_test,pred):.3f}  F1:{f1_score(y_test,pred):.3f}")
```

eta=0.30 (速い・不安定)

→ 精度:0.747 F1:0.709

eta=0.10 (バランス)

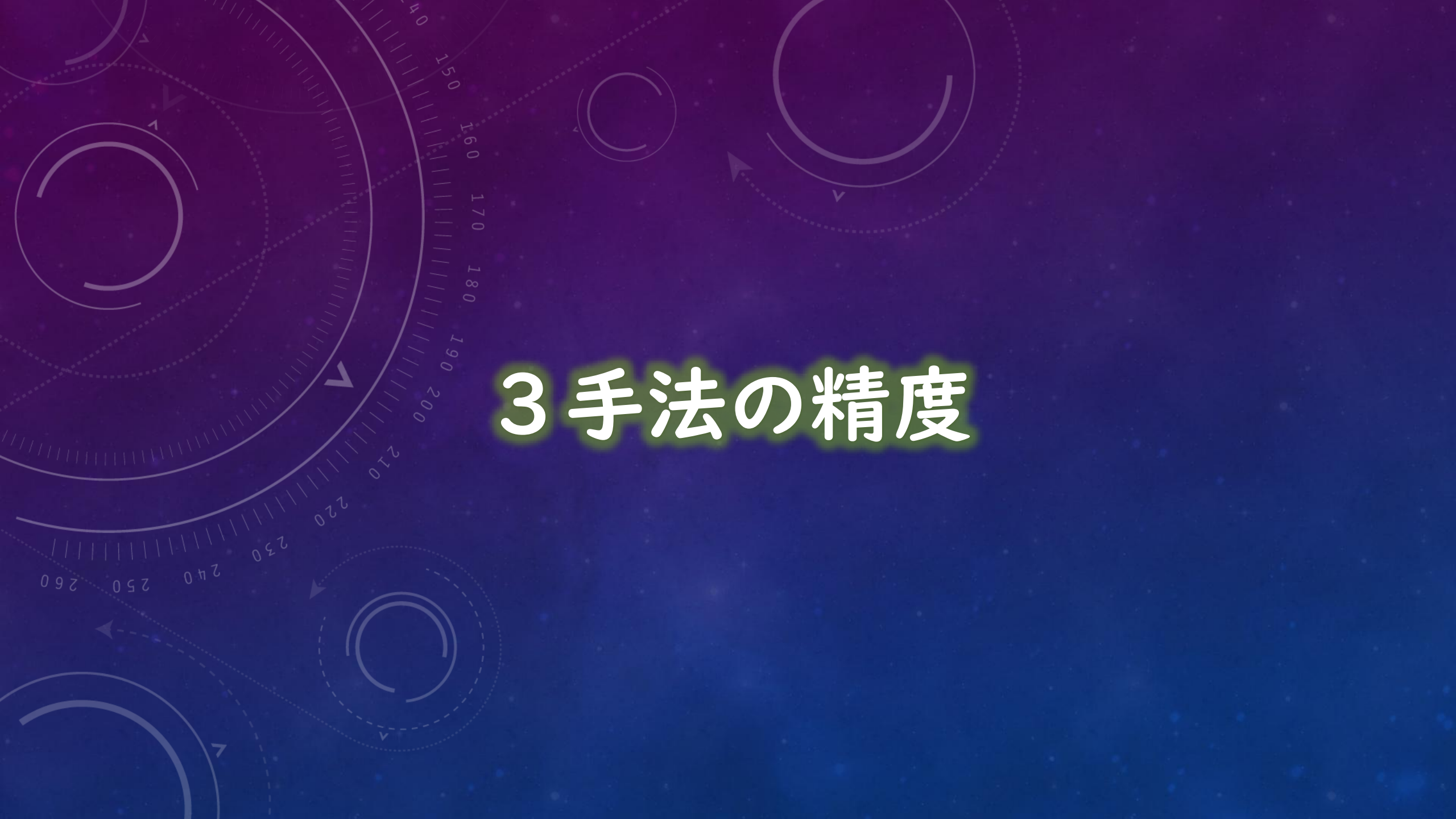
→ 精度:0.751 F1:0.711

eta=0.05 (遅い・安定)

→ 精度:0.753 F1:0.714

eta=0.01 (非常に遅い)

→ 精度:0.749 F1:0.709

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale on the left side features degree markings from 140 to 260 in increments of 10. Other circular patterns, some with arrows indicating direction, are scattered across the frame.

3 手法の精度

3手法の精度比較（Ⅰ）

いろいろ試してみよう！

3手法の精度比較

```
models = {
    "決定木¥n(max_depth=4)": (dt, y_pred_dt),
    "ランダムフォレスト¥n(n=500)": (rf, y_pred_rf),
    "XGBoost¥n(n=200, eta=0.1)": (xgb_model, y_pred_xgb),
}

metrics_data = []
for name, (model, pred) in models.items():
    report = classification_report(y_test, pred, output_dict=True)
    metrics_data.append({
        "手法": name,
        "正解率": accuracy_score(y_test, pred),
        "SH適合率": report["1"]["precision"],
        "SH再現率": report["1"]["recall"],
        "SH F1": report["1"]["f1-score"],
        "macro F1": report["macro avg"]["f1-score"],
    })

metrics_df = pd.DataFrame(metrics_data).set_index("手法")
print(metrics_df.round(3).to_string())

#次に続く。。
```

3手法の精度比較（2）

いろいろ試してみよう！

棒グラフで比較

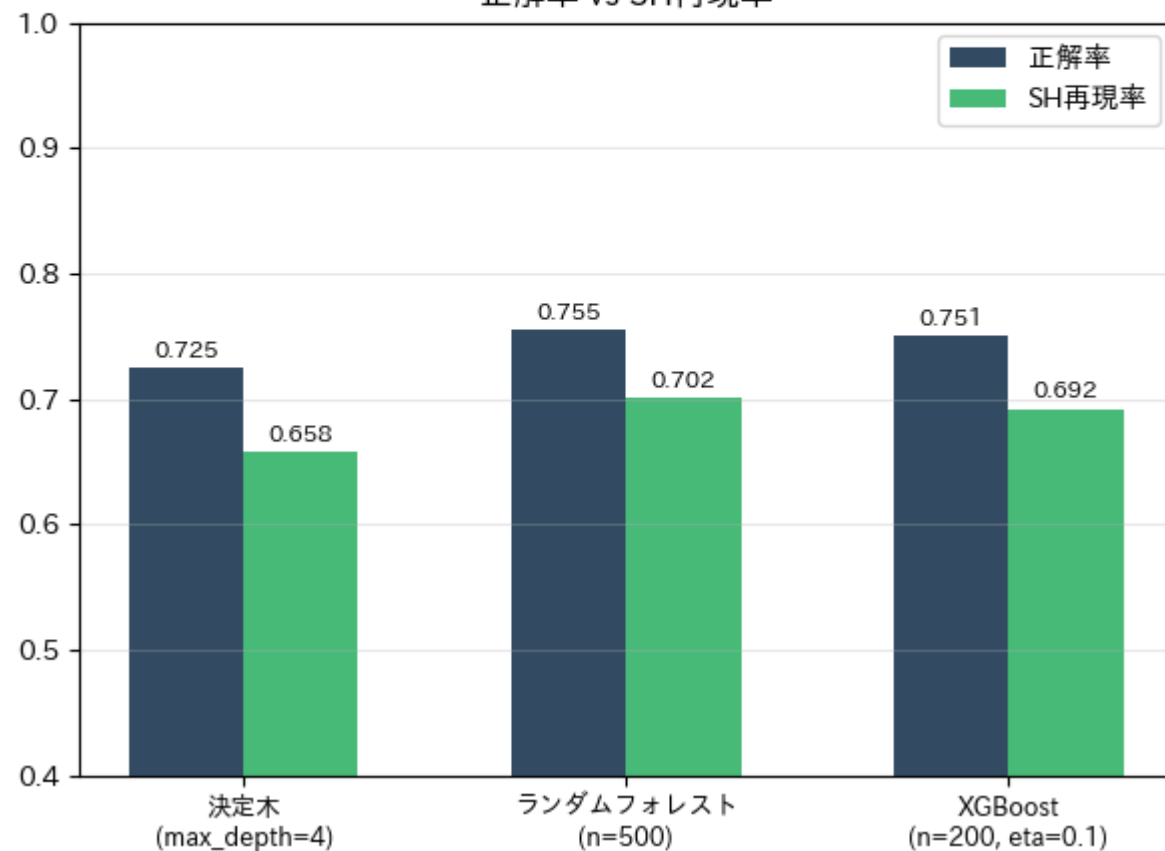
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
metric_pairs = [
    ("正解率", "SH再現率"),
    ("SH F1", "macro F1"),
]
colors = ["#0F2B46", "#27AE60", "#E67E22"]
x = np.arange(len(metrics_df))
w = 0.30
for ax, (m1, m2) in zip(axes, metric_pairs):
    for i, col in enumerate([m1, m2]):
        bars = ax.bar(x + i*w, metrics_df[col], w,
                      label=col, color=colors[i], alpha=0.85)
        for bar in bars:
            ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
                    f"{bar.get_height():.3f}", ha="center", va="bottom",
                    fontsize=8.5)
    ax.set_xticks(x + w/2)
    ax.set_xticklabels(metrics_df.index, fontsize=9)
    ax.set_ylim(0.4, 1.0)
    ax.legend(fontsize=10)
    ax.grid(True, axis="y", alpha=0.3)
    ax.set_title(f"{m1} vs {m2}", fontsize=12)
plt.suptitle("3手法の精度比較(Airbnb Tokyo:スーパーホスト判定)",
            fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("accuracy_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```


3手法の精度比較（3）

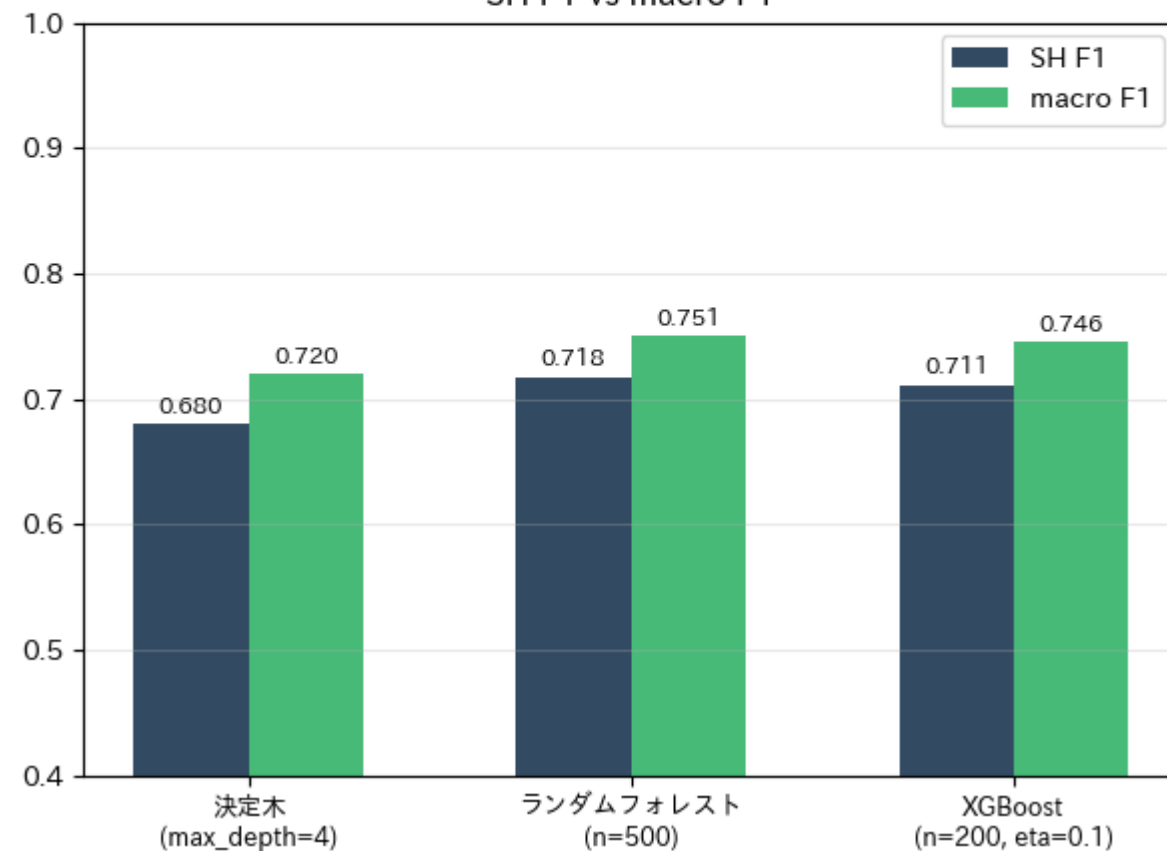
手法	正解率	SH適合率	SH再現率	SH F1	macro F1
決定木(n(max_depth=4)	0.725	0.703	0.658	0.680	0.720
ランダムフォレスト(n(n=500)	0.755	0.735	0.702	0.718	0.751
XGBoost(n(n=200, eta=0.1)	0.751	0.732	0.692	0.711	0.746

3手法の精度比較（Airbnb Tokyo：スーパーホスト判定）

正解率 vs SH再現率



SH F1 vs macro F1



The background is a deep blue gradient with a subtle pattern of white dots. Overlaid on the left side are several concentric circular arcs and a large circular scale with degree markings from 140 to 260. Some of these circles have dashed lines and arrows, suggesting a sense of rotation or movement. The main title is centered in the middle-right area.

特征量重要度

特徴量重要度の比較（１）

いろいろ試してみよう！

特徴量重要度の比較

```
imp_dt = dt.feature_importances_  
imp_rf = rf.feature_importances_  
imp_xgb = xgb_model.feature_importances_
```

```
imp_df = pd.DataFrame({  
    "決定木": imp_dt,  
    "ランダムフォレスト": imp_rf,  
    "XGBoost": imp_xgb,  
}, index=FEATURES)
```

```
print(imp_df.round(3).to_string())
```

横並びの棒グラフ

```
fig, axes = plt.subplots(1, 3, figsize=(14, 5))  
plot_colors = ["#0F2B46", "#27AE60", "#E67E22"]  
plot_titles = ["決定木¥n(max_depth=4)", "ランダムフォレスト¥n(n=500)", "XGBoost¥n(n=200)"]
```

#次に続く。。。

特徴量重要度の比較（２）

いろいろ試してみよう！

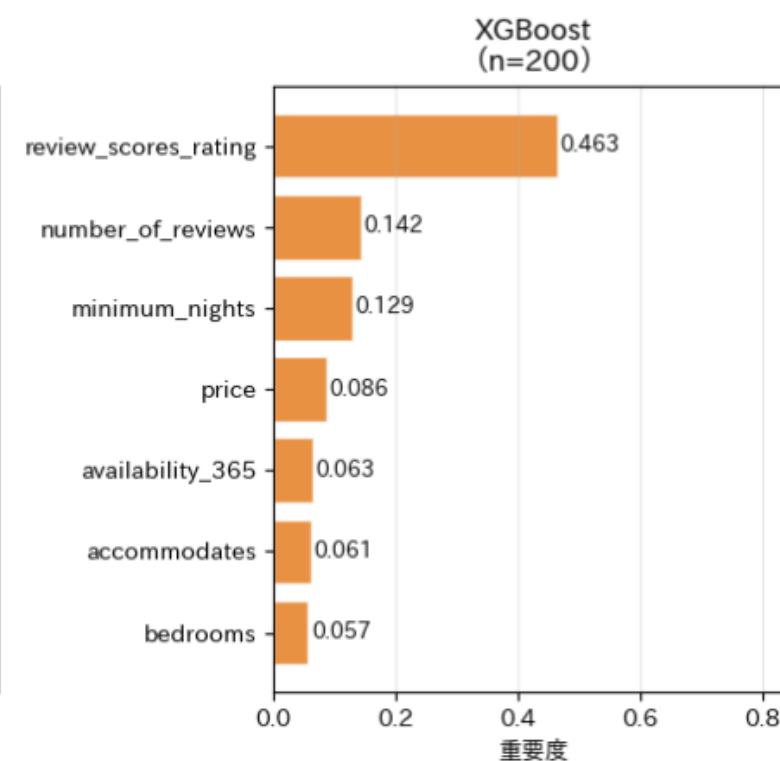
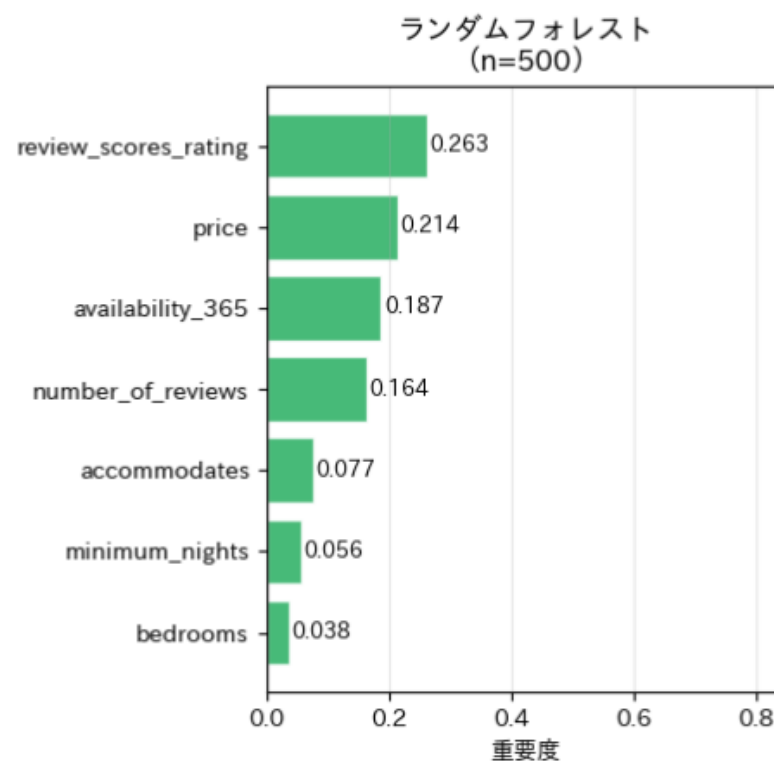
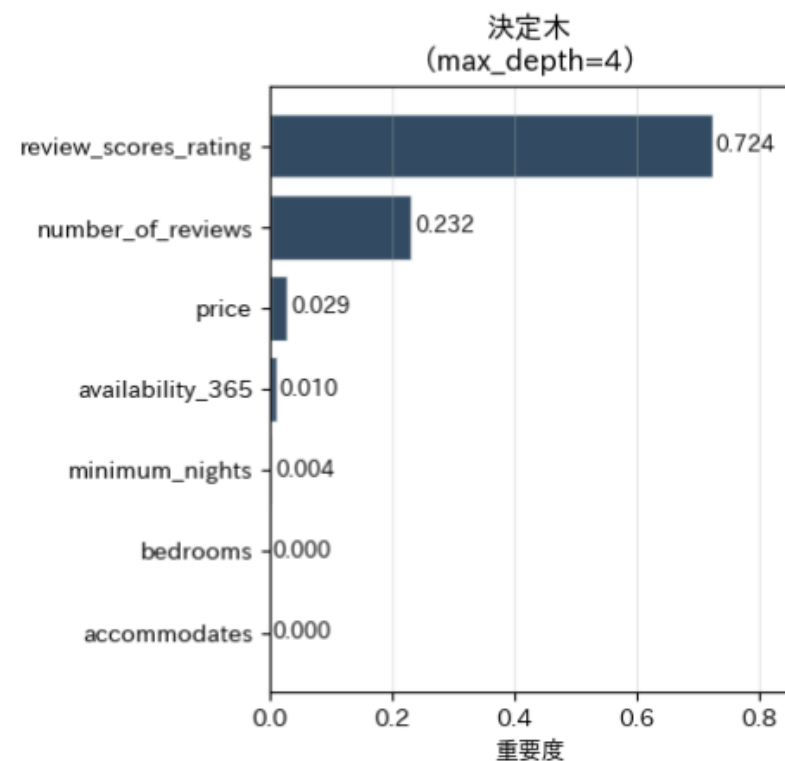
```
for ax, col, color, title in zip(
    axes,
    ["決定木", "ランダムフォレスト", "XGBoost"],
    plot_colors,
    plot_titles
):
    sorted_df = imp_df[col].sort_values()
    bars = ax.barh(sorted_df.index, sorted_df.values,
                   color=color, alpha=0.85, edgecolor="white")
    for bar, val in zip(bars, sorted_df.values):
        ax.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
                f"{val:.3f}", va="center", fontsize=9)
    ax.set_title(title, fontsize=12, fontweight="bold")
    ax.set_xlabel("重要度", fontsize=10)
    ax.set_xlim(0, 0.85)
    ax.grid(True, axis="x", alpha=0.3)

plt.suptitle("3手法の特徴量重要度比較(Airbnb Tokyo)",
            fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("feature_importance_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```


特徴量重要度の比較（3）

	決定木	ランダムフォレスト	XGBoost
price	0.029	0.214	0.086
number_of_reviews	0.232	0.164	0.142
review_scores_rating	0.724	0.263	0.463
accommodates	0.000	0.077	0.061
bedrooms	0.000	0.038	0.057
minimum_nights	0.004	0.056	0.129
availability_365	0.010	0.187	0.063

3手法の特徴量重要度比較（Airbnb Tokyo）



The background is a deep blue gradient with a starry texture. Overlaid on the left side are several concentric circular patterns. A prominent circular scale with degree markings from 140 to 260 is visible. Other circles of varying sizes and line styles (solid, dashed) are scattered across the frame, some with arrows indicating a clockwise direction.

混同行列

混同行列の比較（Ⅰ）

いろいろ試してみよう！

混同行列の比較

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
plot_titles2 = ["決定木", "ランダムフォレスト", "XGBoost"]
pred_list = [y_pred_dt, y_pred_rf, y_pred_xgb]

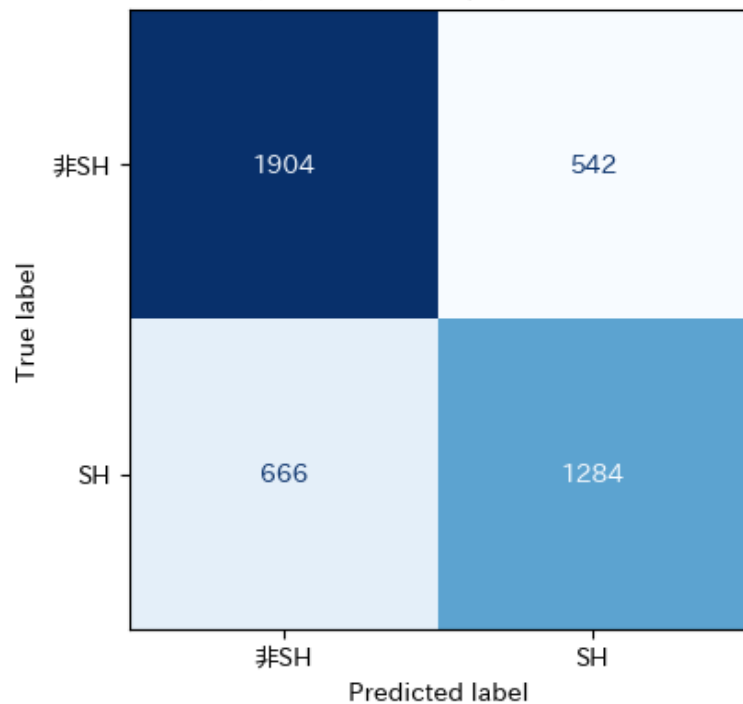
for ax, pred, title in zip(axes, pred_list, plot_titles2):
    cm = confusion_matrix(y_test, pred)
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm, display_labels=["非SH", "SH"]
    )
    disp.plot(ax=ax, colorbar=False, cmap="Blues")
    acc = accuracy_score(y_test, pred)
    sh_recall = cm[1,1] / cm[1].sum()
    ax.set_title(f"{title}\n正解率={acc:.3f} SH再現率={sh_recall:.3f}",
                fontsize=11, fontweight="bold")

plt.suptitle("3手法の混同行列比較(Airbnb Tokyo)",
            fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("confusion_matrix_comparison.png", dpi=150, bbox_inches="tight")
plt.show()
```

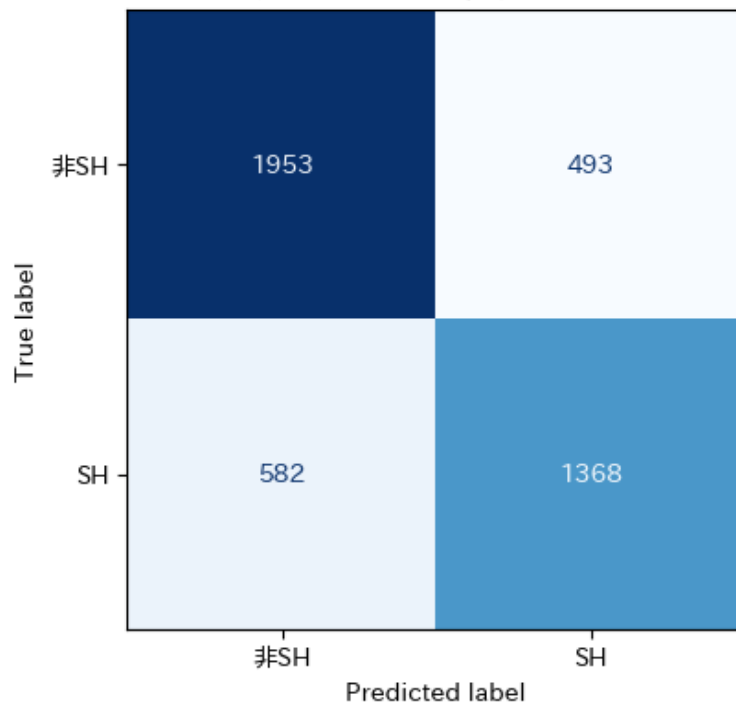
混同行列の比較（2）

3手法の混同行列比較（Airbnb Tokyo）

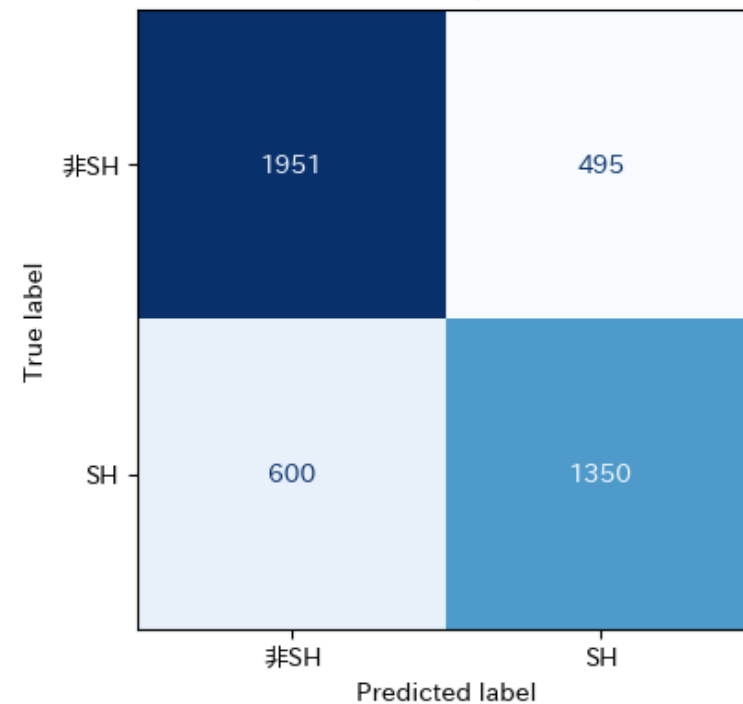
決定木
正解率=0.725 SH再現率=0.658



ランダムフォレスト
正解率=0.755 SH再現率=0.702



XGBoost
正解率=0.751 SH再現率=0.692



The background is a gradient from dark purple at the top to dark blue at the bottom, speckled with small white dots. On the left side, there are several concentric circular patterns. A large circle has a degree scale from 140 to 260. Other smaller circles have partial scales or arrows. The text 'まとめ' is centered in the middle-right area.

まとめ

バギング vs ブースティング

観点	バギング（ランダムフォレスト）	ブースティング（XGBoost）
学習の仕組み	並列・独立して学習	逐次・前の誤りを修正
解決する問題	高バリエーション（過学習）	高バイアス（未学習）
計算速度	◎並列化できる	△逐次のため遅い
過学習リスク	◎低い	△ハイパーパラメータ調整が必要
精度（一般的に）	○高い	◎より高い
外れ値への耐性	◎強い	△弱い（外れ値に引っ張られる）
ハイパーパラメータ	少ない（ntree, mtry）	多い（eta, max_depth等）

実務の目安：まずランダムフォレストで試し、精度不足ならXGBoostへ。
ランダムフォレストは解釈しやすく失敗しにくい。

ブースティングの系譜

年	手法	概要
1995	AdsBoost	ブースティングの元祖。誤分類されたデータの重みを増やして次の木を学習させる。
2001	Gradient Boosting (GBDT)	AdaBoostを勾配降下法で一般化。残差を直接フィッティングするアイデア。
2016	XGBoost	正則化・高速化・欠損値処理を追加。Kaggle優勝の定番手法に。
2017	LightGBM (Microsoft)	リーフワイズの木成長とGOSSで大規模データに対応。実務では最もよく使われる。
2017	CatBoost (Yandex)	カテゴリ変数の自動エンコーディング。前処理コストを大幅削減。

4

本日の課題

頑張ってください。



課題

【課題】 期限：2026/5/27（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第6回（5/21）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```